

1. Planteamiento del Problema

Los fandoms de fantasía/romance y de metal/rock ya cruzan sus mundos de forma orgánica (playlists de fans, referencias temáticas), pero completamente a mano — sin ninguna herramienta que lo sistematice.

Plataforma que recomienda música rock/metal a partir de lo que lees, y libros de fantasía/romance a partir de lo que escuchas.

POC – prototipo acotado a un catálogo curado de bandas/subgéneros de metal-rock (MusicBrainz) y libros de fantasía/romance (Open Library / Google Books).

Nicho de Negocio

Descubrimiento cruzado de contenido para fandoms — intersección metal/rock x fantasía/romance.

Usuario Final

1. Lectores-oyentes de fandom (BookTok/Bookstagram, comunidades de metal)  
2. Editoriales, librerías y sellos de metal (B2B) — cross-selling y contenido

Imprescindible

Ideal tenerlo

Opcional

Se puede omitir

2. Fuente(s) de Datos

MusicBrainz

Gratis, sin llave · ~1 req/seg anónimo (50/seg servidor) · User-Agent propio · metadatos, géneros y subgéneros de rock/metal

Genius API

Gratis, oficial · solo metadatos (título, popularidad) — se descartó el scraping de letras: bloqueado por Cloudflare/ToS

Open Library

Gratis, sin llave · 1-3 req/seg · catálogo y sinopsis de fantasía/romance

Google Books API

Gratis con cuota diaria · respaldo de metadatos de libros

3. EDA, Visualización

Tablas

Catálogo de bandas/canciones  
Catálogo de libros  
Distribución de subgéneros

Gráficas

Ley de Zipf y nubes de palabras (sinopsis)  
t-SNE de embeddings  
Distribución de subgéneros

KPIs

Cobertura de subgéneros  
% catálogo con texto suficiente  
Diversidad de recomendaciones

Stack Tecnológico

spaCy  
gensim  
scikit-learn  
pandas / numpy

plotly  
matplotlib  
seaborn

OpenAI API

Groq / Gemini  
DeepSeek (resp.)

Google Colab

Kaggle Notebooks  
(resp. GPU)

Hugging Face  
Spaces  
(Gradio / Streamlit)

Streamlit  
Community Cloud  
(resp.)

MusicBrainz  
Open Library

Google Books  
Genius API (resp.)

Spotify oEmbed  
(reproductor)

git / github

VS Code / Cursor

Claude Code

4. Ingeniería de Variables

FASE 1 — Limpieza

Limpieza y estandarización de texto (sinopsis de libros) · remoción de stopwords, lematización, detección de frases (gensim) · deduplicación a nivel libro

FASE 2 — Vectorización

Word2Vec (CBOW) / embeddings de oración sobre sinopsis · features estructurales de texto: densidad léxica, TTR, POS, entidades nombradas, pronombres 1a/2a persona (spaCy)

FASE 3 — Variables puente

Vector de género/subgénero de MusicBrainz como proxy de "mood" musical · pares semilla libro↔banda/canción curados a mano para anclar el mapeo cruzado

5. Modelación supervisada/no supervisada

Modelado de tópicos (LDA) sobre sinopsis de libros — temas mapeados después a tags/géneros musicales

Dual encoder supervisado: proyecta sinopsis al espacio semántico de tags/géneros musicales de MusicBrainz, entrenado con pares semilla

Clustering no supervisado en el espacio conjunto — agrupa por "vibra" temática compartida

6. Pipeline / Inferencia

Consulta del usuario

→

Flujo chat

→

Reconocer intención (libro↔música)

→

Generar / recuperar embedding

→

Buscar vecinos en espacio conjunto

→

Llamada a la API de OpenAI

→

Devolver respuesta en UI

Estructurar respuesta

→

Reproductor de sonido (si aplica)

7. Consumo de modelos

API de OpenAI — prompt engineering + vibe engineering

Groq / Gemini / DeepSeek — respaldo si se satura la cuota

ASR (speech2text)

8. Elementos gráficos UI

Chat conversacional

Vista admin / login / landing page

Reproductor de sonido — MusicBrainz→enlace Spotify vía oEmbed